

claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_635348d6-51c\agent-abd8d1431729b6af5.jsonl`
- SHA-256: `9f84b08add10db688a5b16e40afd48bb1c95906ecfc718f1e6019ba51c79de6f`
- Source modified: `2026-06-21T13:56:27+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_635348d6-51c`
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`
```

Transcript

1. user (2026-06-21T13:54:04.237Z)

In the engine repo at C:/Users/avidu/Projects/badminton-highlight-indexer, read the serving roadmap to map the Cloud-Run-friendly milestones. Read docs/COMPUTE_DECOUPLED_SERVING/ (esp. any README / tracker / milestone list ? the source of truth for M5?M12), plus the relevant ADR docs (ADR-011/012/013 ? likely under docs/archives/decisions/ or docs/). Identify which milestones are specifically about making the tool Cloud-Run-friendly (containerize, dispatch to Cloud Run Jobs/Service, scale-to-zero, bucket-poll completion, per-video billing) and each one's STATUS as the tracker marks it (? done / ? in-progress / ? planned / owner-gated). Summarize the Cloud Run serving DESIGN (Jobs vs Service, GPU vs CPU, completion detection, the containerization approach + the WASB torch-1.11 conda-env concern). Return structured JSON.

2. assistant (2026-06-21T13:54:07.168Z)

I'll start by reading the global convention to git pull, then explore the serving roadmap docs.

3. user (2026-06-21T13:54:10.331Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-21T13:54:10.515Z)

```
total 88
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:20 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:23 ..
-rw-r--r-- 1 avidu 197609 33780 Jun 21 19:20 README.md
-rw-r--r-- 1 avidu 197609  8643 Jun 21 08:15 TESTING_STRATEGY.md
-rw-r--r-- 1 avidu 197609 10159 Jun 21 08:17 architecture.svg
drwxr-xr-x 1 avidu 197609    0 Jun 21 15:46 runlogs
```

5. assistant (2026-06-21T13:54:13.431Z)

The pull can't fast-forward (likely a repo config quirk). Per global instructions, I won't force anything and will proceed read-only on local state. Let me read the source-of-truth README and the runlogs/tracker.

6. user (2026-06-21T13:54:15.364Z)

```
1      # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2
3      > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** .
**Owner:** Avi . **Created:** 2026-06-21 . **Last updated:** 2026-06-21
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to
`docs/archives/past_projects/` when DONE)
5      > **Tracking anchors:** $7 progress tracker is the source of truth; pointer in memory
```

```

`current-state` + `docs/NEXT_STEPS.md` once work starts.
6     > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7     > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
**compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an
afterthought.
8     > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10    ---
11
12    > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video
? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next
to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13,
a headline feature), and execution is **never a surprise** ? auto-detect **suggests**, the user
**confirms** (D15).
13
14    ## 0. TL;DR
15    The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three
deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch
on**. A **`ComputeBackend` (deployment profile)** decides **where** the core runs (a local process
vs a Cloud Run GPU job); a **`StorageBackend`** decides **where bytes come from** (local FS / USB
/ mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local**
(local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive /
bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus
**cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the
same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the
**parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.
16
17    ## 1. Problem & goal
18    - **Why now:** DECOUPLED_COMPUTE proved compute **portability** (M0?M2 + M3.5 on a GCP L4)
and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero +
cost guard)** become this project. The owner needs **both** a hosted service **and** a truly-local
mode, with the **core (detect+stitch) unaffected** by which one runs.
19    - **The two user-facing outcomes** (owner, 2026-06-21):
20    - **a) Hosted service** ? operate on small **local uploads (`<2GB`)** **and** load from
**gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching**
(stitching is compute-intensive ? run it where its cost is accounted for).
21    - **b) Truly-local** ? on a local machine with videos on **local drives/USB + a local
GPU**, run inference **and** stitching **completely locally**.
22    - **"Good" looks like:** the same `video ? rallies ? reel` core gives identical windows
+ stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup
choice**, never a code branch in the core.
23
24    ## 2. Decisions locked
25
26    | # | Decision | Source | Implication |
27    |---|-----|-----|-----|
28    | D1 | The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile
branch inside it**. | design `[verified]` | All profile difference lives in config + the two
registries. |
29    | D2 | **Two registries:** `ComputeBackend` (where) selected by `deployment.profile` +
`compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`. | ADR-014
| Realizes PLATFORM_ARCHITECTURE's `local`|`hosted`|`byo_worker`. |
30    | D3 | **Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI);
`byo_worker` reserved/deferred. | ADR-014 | Enum/seam reserved so BYO isn't precluded. |
31    | D4 | In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so
CPU/wall + egress are metered into the per-job cost ledger. | owner (a) | Stitch-on-laptop would
hide real cost. |
32    | D5 | **Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory);
**detect** is byte-exact only same-GPU. | deploy report 2026-06-20 | Cross-GPU asserted at the
rally/window level, never CSV bytes. |

```

33 | D6 | ****Default-OFF, smallest-safe-first, one PR per milestone****; any core-caller change (e.g. configurable output base) ships behind a flag + a Launch Report. | **[[launch-report-convention]]** | Zero-regression discipline. |

34 | D7 | ****Cloud Run cold-start: SCALE-TO-ZERO**** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). ****No warm pool**** (that ~\$500/mo idle would break per-video billing). | owner 2026-06-21 | M6/M7. Revisit a warm lane only on a UX complaint. |

35 | D8 | ****Pricebook = a versioned DB table****; cost computed in ****USD**** (matches GCP billing = one source of truth), ****displayed in INR**** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change. | owner 2026-06-21 | M8. |

36 | D9 | ****Edge auth = Cloudflare Access**** (reuse the site's existing CF). Lock the Cloud Run ****ingress to the CF proxy**** + ****verify the `Cf-Access` JWT signature**** (so a direct hit to the Cloud Run URL can't spoof the identity header). | owner 2026-06-21 | M9. One identity system. |

37 | D10 | ****Free-tier "data-for-reels" trade:**** free reels in exchange for ****training rights**** (uploaded footage + derived trajectories/labels); ****paid tier = no-train**** (privacy is the paid feature). ****Carve-outs:**** train ONLY on attested ****ADULT**** footage (minor footage ? reel yes, training pool ****NO**** ? DPDP/GDPR need verifiable parental consent); train on ****DERIVED**** data + ****auto-delete raw****; ****ToS counsel-reviewed****; live training-on-uploads ****gated to M9**** (public signup). Truly-local needs no DPA. | owner 2026-06-21 | M9 + D12; ties **[[paper-coauthor-aim]]** / PERSONALIZATION flywheel. |

38 | D11 | ****Quality floor: Gemini handoff ON**** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a ****Launch Report****; the owned model runs in ****shadow/eval**** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.) | owner 2026-06-21 | M7 + D12. |

39 | D12 | ****Data-flywheel harness is IN SCOPE (owner add, Q5):**** capture the (consented, adult) uploaded video + the Gemini reel/rally output ? ****reliably store**** in the per-video workspace/bucket ? ****run shadow evals**** (owned-vs-Gemini, dual-eval-set) ? ****promote good ones to the golden TRAINING set**** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off). | owner 2026-06-21 | new ****M11****; ties D10 (consent) + **`data\_pipeline/`** + the eval/experiment harness. |

40 | D13 | ****Profile is user-switchable ANYTIME ? a headline FEATURE.**** The same user runs ****local today, cloud tomorrow, back to local**** ? it's just a config/startup choice; the pure core gives ****equivalent**** output either way. ****Advertise it**** ("your machine *or* our cloud ? your call, switch anytime"). | owner 2026-06-21 | Honest caveat: equivalent at the ****rally/window**** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is ***between*** jobs. |

41 | D14 | ****NORTHSTAR ? operate toward this:**** a ****mobile, app-like**** flow ? point at a ****Google Drive**** video ? ****fully cloud-native**** run ? the stitched, ****ready-to-share**** reel lands ****back in Drive, next to the source****. | owner 2026-06-21 | Implies a ****`gdrive-api` (OAuth) StorageBackend**** (read source + write reel back) ? distinct from the local ****mount**** backend; ****resolves \$8 #7****, new ****M12****. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the **`StorageBackend`** ABC accommodates it). |

42 | D15 | ****Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.**** A GPU owner may still choose cloud; ****cloud (= spend) is never entered silently****; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends \$4.3 (auto-detect = a proposed default, not an auto-decision). |

43 | D16 | ****M6/M7 implementation SHAPE + the cheap \$8 picks (owner 2026-06-21):**** (a) ****Cloud Run JOBS**** (not Services-with-GPU) for the detect+stitch unit, ****bucket-poll for completion**** (reuses **`JobWaiting`/`claim\_due\_job`** verbatim, fits async upload?poll?reel + scale-to-zero D7); (b) ****stitch ENCODE = libx264**** (protects the D5 byte-determinism/parity guarantee + the L4 parity test) ? ****NVDEC for DECODE only**** (a speed lever that never touches output bytes); ****NVENC** encode is DEFERRED behind a config knob**, revisit only if M7 cost data justifies relaxing parity (\$8 Q8); (c) ****input cap = hard-reject `<2GB`**** for v1 (a config cap so raising to 4GB later is one line; matches the shipped M2 chunked-upload threshold ? \$8 Q9). | owner 2026-06-21 | M6/M7. Resolves \$8 Q2 (co-located, = D4), Q8, Q9, Q10 (dir name confirmed). |

44 | D17 | ****Real cloud verification BUDGET = \$100 / ?10k cap (owner 2026-06-21):**** the owner authorized actual GCP spend for due-diligence + production-grade testing/verification of M5?M7 (real L4 run, the M5 runlog, the M7 DEPLOY_REPORT), ****provided the cap is respected**** ? confirm exact cmd + hourly cost before each spend, prefer the smallest viable run + cheapest-adequate GPU, and ****DELETE every resource after**** (**[[gcp-vm-always-delete]]**). Removes the M6/M7 "spend" gate within the envelope; the calibrated pricebook (M8) + counsel ToS (M9) + OAuth app (M12) stay owner/counsel-side. | owner 2026-06-21 | Unblocks the real M5/M7 runs (not just the

```

code) within the cap. |
45
46     ## 3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end*
47     This project is the latest step in a long decision chain; each ADR contributed a piece
and a **subtlety that carries forward**:
48
49     | ADR / doc | Contributed | Subtlety carried into this project |
50     |---|---|---|
51     | **ADR-005** | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served
model + the **container image** (ffmpeg, fonts, weights) stay commercial-clean. |
52     | **ADR-008** | Owned-model topology; **train==serve featurizer single-sourcing** | The
"core unaffected across profiles" is the same parity discipline, extended from train/serve to
**local/cloud**. |
53     | **ADR-009** | KhelSutra Guru; **local-first delivery** | The **truly-local profile**
is the direct continuation ? privacy/offline self-host stays first-class. |
54     | **ADR-010** | DECOUPLED: DO?GCP pivot | The cloud-compute origin (GPU + co-located
bucket). |
55     | **ADR-011** | DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4
(billing/cost-guard)**; overrode the **rent nothing / not a service** posture | **This project
realizes M3/M4.** Carries $06's owner-gated gates: **DPA/consent for non-owner uploads,
collector `md5` keying, WASB-C3**. |
56     | **ADR-012** | GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving
model**; **NVDEC** named as the decode lever | The serving substrate + the 7-step seed scope
this expands. |
57     | **ADR-013** | Drop D0; **GCP sole compute stack**; $200 D0 credits ? non-compute only
| The provider for cloud-serving; D0 Spaces remains only a possible S3 *storage* target. |
58     | **? ADR-014 (new)** | **Compute-decoupled serving:** one pure core, profile-selected
compute+storage; stitch-where-metered | This doc. **Refines** ADR-011/012, does **not**
supersede them. |
59     | PLATFORM_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The
abstraction this realizes in code. |
60     | VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md | L0 fully-local tier / `LocalDesktop`
backend | The truly-local profile = these, productionized. |
61     | 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB
hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes. |
62     | DEPLOY_REPORT_GCP_2026-06-20 (archived) | First real L4 run; **cross-GPU sub-pixel
parity finding** | Why D5 (parity at window level, not CSV bytes). |
63
64     ## 4. Design / architecture
65     See [architecture.svg](architecture.svg) for the picture. **The core never changes;
the box around it does.**
66
67     ### 4.1 The core contract (must stay byte/behaviour-identical) `[verified]`
68     1. **DETECT (GPU):** `DetectorRunner.run_predict(video_path, output_dir, video_id) ?
CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable
impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ?
emit the **identical CSV schema**. The trajectory CSV is the stable artifact boundary.
69     2. **WINDOW (CPU pure fn):** `trajectory_to_action_windows(points, fps, frame_width, ?)
? windows` (`tracknet_runner.py:281`). A shared `@staticmethod`, zero GPU/IO/randomness ? same
trajectory + config = byte-identical candidate windows. Used by every runner *and* the
eval/regression stack verbatim.
70     3. **STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments,
out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffmpeg ? watermark
? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.
71
72     **Non-goal (the trap to avoid):** *no* code branch inside detect/window/stitch keyed on
profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-
identical to CONTROL).
73
74     ### 4.2 The deployment profiles
75     | Profile | Compute | Storage | Orchestration | When |
76     |---|---|---|---|---|
77     | **truly-local** | `compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto`
(WSL); stitch **in-process** | `local` (or `gdrive` = mounted Drive) | `python -m backend.main`
or `worker infer` CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-

```

```

box ? owner (b) |
78 | **cloud-serving** | `compute_target=cloud_run`; `detector_impl=native` (L4, cuda);
**detect AND stitch on the same Cloud Run job** | `gcs` (or gdrive mount / assembled upload);
per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job
pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero | Hosted
SaaS ? owner (a) |
79 | **cpu-mock** | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) |
`local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free
dev |
80 | *byo_worker* (deferred) | tenant GPU via outbound pull agent | tenant's own bucket
(signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ?
**DEFERRED**, enum/seam reserved |
81
82 ### 4.3 Profile selection (the "startup/setup config") `[design]`
83 ONE new top-level `deployment` section on `AppConfig` + preset application + env auto-
detect in `load_config`:
84 - `deployment.profile` ? {truly-local | cloud-serving | cpu-mock} (empty = today's
manual knobs, **fully backward-compatible**).
85 - Selecting a profile applies a **coherent preset bundle** of *existing* knobs that
today drift independently: **INPUT** (new `input_backend`/`input_root`, parallel to the output-
side `StorageConfig`), **COMPUTE** (new `compute_target` enum locking `detector_impl` +
`skip_ai_handoff` + workspace strategy), **STORAGE**
(`storage.backend`/`workspace_root`/`single_folder_per_video`, all already exist).
86 - **Precedence:** built-in defaults ? profile preset ? `config.json` ? env
(`RALLY_DETECTOR_IMPL`, `WASB_*`, `DEPLOYMENT_PROFILE`, ?) ? worker `--set k=v`. Every knob
stays individually overridable.
87 - **Env auto-detect SUGGESTS, the user CONFIRMS (D15 ? no surprise execution): **the env
gives a *proposed* default (`K_SERVICE` ? cloud-serving; `CI=true` ? cpu-mock; else
`torch.cuda.is_available()` ? truly-local), but the user **explicitly confirms or overrides** it
? a GPU owner may still pick cloud, and **cloud (= spend) is NEVER entered silently**. The
active profile is always surfaced (logged/shown). **Per-profile startup checks fail/warn fast**
(cloud-serving without GCS creds/GPU ? loud error before any job).
88 - The worker (`cmd_infer`/`cmd_train`) is **already profile-agnostic** (reads
`config.storage`, accepts `--config`/`--set`) ? no worker rewrite.
89
90 ### 4.4 Compute placement ? and *why stitch runs on Cloud Run* `[design]`
91 DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In
**cloud-serving, stitch is co-located in the Cloud Run job** so `gpu_active_seconds` (detect) +
`wall_seconds` (detect+stitch) + `bytes_out` (reel egress) land on **one metered invocation** ?
the per-job cost ledger. Running stitch on a laptop would leave that cost
**invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in
the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get
bursty ? the job table + `claim_due_job` already supports both.)
92
93 ### 4.5 Reuse map ? most of the seams already exist `[verified]`
94 **Exists:** the 3-stage core; `_resolve_runner` (the single compute seam);
`StubDetectorRunner` (`replay_csv` = byte-exact $0 anchor); `StorageBackend` ABC + 4 backends +
`StorageRef.parse_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control
plane (`jobs` table, `claim_due_job` CAS); `ExecutionPolicy WAIT_AND_RESUME`/`JobWaiting`
(carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip_ai_handoff`
(LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video
workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user_id` columns
exist). **Net-new:** the `deployment` section; `input_backend`; configurable output base (kill
hardcoded `output`/); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-
auth header extraction; startup env detection.
95
96 ### 4.6 The data-flywheel harness (D12 / M11) `[design]`
97 Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip
Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline
additionally:
98 1. **Captures** the source upload + the Gemini reel + the rally output
(intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-
consent auto-delete path.
99 2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw
video kept only for the training pool, not the no-consent path).

```

100 3. ****Runs shadow evals**** ? the owned model scores the same video alongside Gemini
(`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks
the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").

101 4. ****Promotes to golden**** ? captured videos that pass review become ****golden TRAINING
rows**** (human-reviewed labels via the annotation flow ? `data\_pipeline/`), growing the corpus ?
the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.

102 This is the consent?capture?eval?goldenize ****flywheel**** (the cloud realization of the
PERSONALIZATION contribution model): every consented free reel makes the owned model better.
Reuses `data\_pipeline/`, `backend/eval/`, and the M9 consent gate; ****adults-only****, raw-
retention gated by the D10 consent.

103

104 ## 6. Honest limits ? what this does NOT do / prove

105 - A green CI suite proves ****plumbing + determinism (window+stitch) + profile-parity****
for \$0 ? it does ****NOT**** prove field quality, nor that real-GPU detection is good (that's owner-
side, real-video).

106 - ****Cross-GPU detection is NOT byte-identical**** ? parity is a window-level claim, not a
CSV-byte claim.

107 - The cloud `0.1.0-l4` weights are ****n=2 plumbing-grade**** ? cloud-serving ships with
****Gemini handoff ON**** until the owned model clears a floor (open Q).

108 - This does not resolve ****WASB-C3**** (sharing a trained model) or the ****DPA/consent****
gate ? both remain owner/counsel-gated before public, non-owner serving.

109

110 ## 7. Deliverables & progress tracker ? ****source of truth****

111 Legend: ? Todo · ? In progress · ? Done · ? Gated. ****One small PR per row****, off
`origin/master`, no stacking. Default-OFF where it touches core callers.

112

113 | ID | Deliverable | Depends | Gated? | Status | PR |

114 |----|-----|-----|-----|-----|----|

115 | M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? |

116 ****? APPROVED 2026-06-21**** |

117 | M1 | `deployment.profile` + `compute\_target` config + preset bundles +
precedence/auto-detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? |

118 [#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |

119 | M2 | `input\_backend`/`input\_root`; wire main CLI/API input through
`StorageRef.parse\_uri` (local=identity); storage-fake tests | M1 | safe | ? |

120 [#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280) |

121 | M3 | Configurable output/scratch base ? kill hardcoded `output/`
(`trajectory\_hybrid:237,313`, `yolo\_hybrid:407`, `gemini`); ****DEFAULT stays `output/`****; golden
+ stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? |

122 [#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |

123 | M4 | `DeviceContext` + WASB ****CPU-fallback****; unified healthcheck/device wiring | M1 |
safe (cuda default unchanged) | ? | [#283](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/283) |

124 | M5 | ****truly-local profile**** end-to-end (preset + startup checks); first committed
****runlog**** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? |

125 [#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286) (preset+startup-
checks+L4 parity ?; runlog ? owner GPU) |

126 | M6 | Cloud Run GPU ****dispatch shim**** (control plane ? job via storage ? re-claim) +
****containerized worker image**** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner
(GCP spend) | ? | [#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)
(shim+image+mocked ?; real build/run = M7 ? owner) |

127 | M7 | ****cloud-serving profile**** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch
on Cloud Run ? reel via signed URL; ****DEPLOY_REPORT**** (posterity, sample runs + contact sheet) |
M6 | ? owner (spend) | ? | ? |

128 | M8 | Per-job ****cost ledger**** (v6 migration: `owner_id, gpu_active_seconds,
wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + ****pricebook**** + aggregation +
`api/.../cost` | M7 | ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/288) (ledger+pricebook+/cost ? default-OFF; real prices + usage-emission
? owner/follow-up) |

129 | M9 | ****Edge auth**** (Cf-Access extraction) + per-tenant scoping on `user\_id`;
DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | ? |

130 | M10 | `byo\_worker` / zero-infra-BYO ? ****design-only, DEFERRED**** | M8,M9 | ? explicit
owner approval | ? | ? |

131 | ****M11**** | ****Data-flywheel harness (D12):**** on a consented adult job, ****capture**** the
upload + Gemini reel/rally output ? ****reliably store**** under the per-video workspace ? ****shadow-**

```

eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the
golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the
owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent
gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |
127 | **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the
source from + writes the stitched reel **back to the user's Google Drive next to the source**
(per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local
mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
128
129 **Testing strategy is a first-class deliverable** ?
[`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ?
[`runlogs/`](runlogs/).
130
131 ## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)*
132 **? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:**
133 1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).
134 3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).
135 4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to
proxy).
136 5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data;
counsel-gated to M9).
137 6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until
the owned model clears the floor; capture?store?eval?goldenize meanwhile).
138
139 **? The remaining smaller picks are LOCKED (owner, 2026-06-21) ? see §2 D16:**
140 2. ? **Stitch placement ? co-located** in the detect job (= D4; one metered unit; the
job table supports a future CPU-split non-breaking, revisit only on bursty-compile metrics).
141 7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the
source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ?
distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
142 8. ? **Codec ? D16:** **libx264 ENCODE** (protect parity) + **NVDEC DECODE-only**
(speed, no output-byte change); NVENC-encode deferred behind a config knob.
143 9. ? **Input cap ? D16:** **hard-reject`<2GB`** for v1, as a config cap (raise to 4GB
later in one line).
144 10. ? **Directory name ? `COMPUTE_DECOUPLED_SERVING/` CONFIRMED final** (already the
name everywhere + in ADR-014); `CLOUD_RUN_GPU_SERVING/` dropped.
145
146 ## 9. Definition of done
147 - [ ] M1?M4 land (config + input + output-base + device layer), each green on golden
cross-OS + experiment no-regression.
148 - [ ] **Profile-parity test green** ? truly-local vs cloud-mock produce byte-identical
windows + segment ranges.
149 - [ ] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog /
DEPLOY_REPORT** with a real sample run.
150 - [ ] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth
(M9) gates multi-tenant.
151 - [ ] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented
adult jobs; the owned model is on a measured path to the D11 floor.
152 - [ ] ? flip Status DONE, archive to `docs/archives/past_projects/`.
153
154 ## 10. References
155 **ADR lineage:** ADR-005/008/009/010/011/012/013 ? **ADR-014**
(`docs/archives/decisions/DECISIONS.md`). **Strategy:** `PLATFORM_ARCHITECTURE.md`,
`HOSTING_PLAN.md`, `VIDEO_LOCALITY_MODEL.md`, `DESKTOP_APP_PLAN.md`,
`LOCAL_APPLIANCE_ARCHITECTURE.md` (kheIsutra). **Archived predecessor:**
`docs/archives/past_projects/DECOUPLED_COMPUTE/` (esp. `05-COST-ATTRIBUTION.md`, `07-COMPUTE-
ABSTRACTION.md`, `06-RISKS-AND-OPEN-QUESTIONS.md`, `DEPLOY_REPORT_GCP_2026-06-20.md`). **Code
anchors`[verified]`:** `backend/pipeline/detectors/base.py:164`, `tracknet_runner.py:281`,
`stitcher/compiler.py:303`, `segmenters/trajectory_hybrid.py:65,237`, `backend/storage/*`,
`backend/worker/__main__.py:88`, `backend/api/{uploads,job_worker}.py`,
`backend/infrastructure/database.py`. **Design provenance:** workflow `wf_56274ff0` (4 code-
mappers ? synthesis; 58 findings).
156
157 ---
158 ### Changelog

```

159 - 2026-06-21 ? created from the design workflow output; M0 in progress.

160 - 2026-06-21 ? ****M0 APPROVED****; ****M1 landed****
 ([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)): ``deployment.profile` + `compute_target` + preset bundles + precedence/auto-detect-as-suggestion (D15) on `AppConfig``. Pure/additive, ``profile`` default == today's behaviour (zero core change); presets touch only existing knobs (no M2 ``input_backend`` / M3 output-base over-reach). Suite 1068 green, cov 84.11%.

161 - 2026-06-21 ? ****M2 landed**** ([#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280)): ``input_backend`/`input_root` + URI-aware main API/CLI input via `resolve_input_uri`/`acquire_local_input`` (local = identity passthrough, byte-for-byte unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; ``local://`` accepted on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report provenance. Suite 1105 green, cov 84.84%.

162 - 2026-06-21 ? ****M3 landed**** ([#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282)): configurable output/scratch base via ``backend/utils/paths.output_base(config)`` ? the 3 segmenters' chunk/handoff/temp dirs root under ``output.output_dir`` (default ``"output"``, byte-identical; default-OFF ? no Launch Report). Fixes the latent bug where scratch ignored ``--output-dir``. Suite 1114 green, cov ~84.9%.

163 - 2026-06-21 ? ****M4 landed**** ([#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283)): ``DeviceContext` + native WASB **CPU-fallback** via a new `device="auto"` (cuda-if-available-else-cpu); explicit `"cuda"` stays strict (no silent slow-CPU downgrade), default unchanged. `"auto"` resolves before argv (never reaches `wasb_infer.py`); CUDA probe cached. Device-selection tested offline; real CPU-inference run owner/GPU-verified. Suite 1126 green, cov 84.96%. **? M1?M4 (the config/input/output/device layer) COMPLETE.**`

164 - 2026-06-21 ? ****M5+ batch authorized + design picks locked**** (owner): the remaining \$8 picks resolved ? ****D16**** (M6 = Cloud Run ****Jobs**** + bucket-poll; ****libx264 encode + NVDEC decode-only****; ****hard-reject <2GB**** input cap; dir name confirmed) and ****D17**** (real cloud verification budget = ****\$100/?10k cap****, delete-after). Owner gave a blanket go to write+merge the ****default-OFF code**** for every remaining gated milestone (M5,M6,M8,M9-auth,M9-consent-cols,M11-plumbing,M12), one isolated PR each, reserving the gate for the real run/spend/calibration/counsel. Grounded by gate-map workflow ``wf_4849400f``.

165 - 2026-06-21 ? ****M8 code landed**** ([#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288)): per-job ****cost ledger**** ? ``backend/billing.py`` (pure ``usage × pricebook`` math, USD internal / INR display) + a ****v6 migration**** (NULLABLE cost columns on ``jobs`` + a versioned ``pricebook`` table seeded ``placeholder-v0`` at \$0; additive-NULl, idempotent) + ``record_job_cost`/`get_job_cost`/`get_video_cost` + `GET /api/jobs/{id}/cost` + `/api/videos/{id}/cost` + `BillingConfig`` (****default-OFF****) + a gated ``_maybe_record_job_cost`` hook. Adversarial review (``wf_498b24af``) folded a ****major silent-GPU-under-bill**** (an unpriced ``gpu_type`` is now surfaced, not invisible \$0) + the ****honest no-op**** caveat (usage emission is a follow-up ? cost 0 until wired) + breakdown reconciliation. Suite green, ruff+mypy clean. ****? M8 ? ? owner inserts real GCP prices (a new pricebook version) + the worker usage-emission is a follow-up; placeholder ships \$0 = no user-facing change. v6 is the shared bump (M9-consent cols extend it).****

166 - 2026-06-21 ? ****M6 code landed**** ([#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)): ``backend/compute/`` ComputeBackend registry ? ``CloudRunCompute`` dispatches a Cloud Run ****Job**** (injectable client) then ****bucket-polls**** a done-marker via ``execution_policy.poll_until`` + the storage layer (D16); ``get_compute_backend`` returns a backend ONLY for ``compute_target=cloud_run`` (else ``None`` ? the worker's in-process path, ****default-OFF****). The dispatched container runs ****inline**** (forced ``local_gpu``+native, never re-dispatch); the worker writes a ``--done-uri`` completion marker on success.
``deploy/cloudrun/{Dockerfile,README}`` = the L4 image (CUDA+ffmpeg+fonts+`wasb` conda env; weights staged per WASB-C3) + the M7 runbook. Adversarial review (``wf_87b22fa4``) folded a ****blocker**** (cloud ``fetch`` needs a dst ? reads silently returned ``{}`` on gcs/s3) + a poll-wait test gap + ``PolicyTimeout``?clean rc 4. Suite green, ruff+mypy clean. ****? M6 ? ? the real build/push/L4 run + DEPLOY_REPORT is M7 (owner, D17 budget).****

167 - 2026-06-21 ? ****M5 code landed**** ([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)): ``backend/config/startup.py`` ? per-profile fail/warn-fast startup checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner healthcheck stays the authority), wired into the server (``_enforce_startup_or_exit`` ? exit 1) + the worker (``cmd_infer`/`cmd_train`` ? rc 2), default-OFF (probe gated behind an active profile ? the worker stays torch-free at ``profile=""``). ****L4 profile-parity test**** (``tests/test_profile_parity.py``): same stub trajectory ? byte-identical windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review folded (``wf_34671ac1``). Suite green, ruff+mypy clean. ****? M5 ? ? the real-GPU truly-local runlog (L5) is owner-side**** (template pre-staged in ``runlogs/``).

7. user (2026-06-21T13:54:15.442Z)

```
total 12
drwxr-xr-x 1 avidu 197609    0 Jun 21 15:46 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:20 ..
-rw-r--r-- 1 avidu 197609 2226 Jun 21 08:15 README.md
-rw-r--r-- 1 avidu 197609 2649 Jun 21 15:46 RUNLOG_truly-local_TEMPLATE.md
```

8. assistant (2026-06-21T13:54:18.428Z)

I have the full source-of-truth tracker. Now let me read the ADR docs (ADR-011/012/013/014) to capture the serving-model ratification and owner gates, and confirm the WASB conda-env concern in the Dockerfile.

9. user (2026-06-21T13:54:20.031Z)

```
551:## ADR-011: DECOUPLED_COMPUTE scope = decoupling core (M0?M2 + M3.5); defer serving/billing;
DO=training, GCP=serving
553:**Date:** 2026-06-20 . **Status:** RATIFIED (owner, in-session). **Refines ADR-010.** ?
**The DO=training / GCP=serving split is SUPERSEDED by [ADR-013](#adr-013-drop-digitalocean-for-
compute--gcp-is-the-sole-compute-stack-training--serving)** ? DigitalOcean is dropped for
compute; GCP is the sole compute stack. (The scope decision ? DECOUPLED = M0?M2 + M3.5 ? still
stands.)
598:## ADR-012: Compute = GPU-native (L4/T4); TPU deferred; Cloud Run + GPU (scale-to-zero) is
the supported serving model
600:**Date:** 2026-06-20 . **Status:** RATIFIED (owner, in-session). **Refines ADR-011**
(concretizes the "GCP = serving" path).
611:  with `gs://khelsutra-rally-corpus`. This is the concrete form of ADR-011's deferred
**M3/M4**.
635: guard)** from ADR-011 become a **new tracked subproject ? "Cloud Run + GPU serving"** ? to
be spun
637: (owner sequencing). Seed scope below. **? REALIZED in ADR-014 (2026-06-21):** the
subproject is
639: (not just cloud serving); ADR-014 expands this 7-point seed into the M0?M10 plan.
661:## ADR-013: Drop DigitalOcean for compute ? GCP is the SOLE compute stack (training +
serving)
663:**Date:** 2026-06-20 . **Status:** RATIFIED (owner, in-session: "drop DO and document it?
record it and bake it for all future work"). **Supersedes the DO=training split in ADR-011;
refines ADR-010 + ADR-012.**
667:1. **DigitalOcean is DROPPED as a compute provider.** **GCP is the sole compute stack** for
BOTH training and serving. (ADR-011's "DO=training, GCP=serving" split is retired ? GCP does
both. The DECOUPLED scope decision in ADR-011 ? M0?M2 + M3.5 ? still stands.)
668:2. **Standing default for ALL future compute work = GCP** (a GPU VM today via `deploy/gcp/`;
Cloud Run+GPU for serving per ADR-012). Do **not** re-evaluate DO/Paperspace for compute without
a new ADR.
673:- **DO GPU is neither enabled on the account nor a clean free-credits path.** The DO API
`/v2/sizes` returns **0 GPU sizes** (GPU Droplets not enabled); GPU Droplets are **North-
America-only** (NYC2/TOR1/ATL1/RIC1/AMS3 ? no India/Singapore); and DO's own docs state that for
**newer customers DO charges the team's primary payment method** when GPU Droplets are created
(anti-abuse) ? so GPU spend hits the **card**, not the promo credit. The ADR-011 premise ("200
credits fund DO training") does **not** hold.
683:- **Capacity reality:** GCP GPU stockouts are common (asia-south1/Mumbai was fully stocked-
out 2026-06-20 ? the M2 run landed an L4 in asia-southeast1/Singapore; weights still land in the
asia-south1 bucket). Sweep zones/regions; scale-to-zero (ADR-012) is part of why Cloud Run+GPU
fits.
686:## ADR-014: Compute-decoupled rally serving ? one pure core, profile-selected compute +
storage
688:**Date:** 2026-06-21 . **Status:** ? **RATIFIED (owner, 2026-06-21 ? M0 design sign-off;
build M1+).** **Refines ADR-011 + ADR-012** (it *realizes* their deferred M3/M4; it does **not**
supersede them); **builds on ADR-013** (GCP sole compute stack). Full design:
`docs/COMPUTE_DECOUPLED_SERVING/`.
691:`ADR-005` (permissive licensing ? engine + image stay MIT/Apache/BSD) . `ADR-008`
```

(train==serve featurizer single-sourcing ? the parity discipline this extends to local?cloud) .
 `ADR-009` (**local-first delivery** ? origin of the truly-local profile) . `ADR-010` (DECOUPLED
 DO?GCP) . `ADR-011` (**deferred M3 serving + M4 billing**; overrode "rent nothing / not a
 service"; \$06 gates = DPA/consent, collector `md5`, WASB-C3) . `ADR-012` (**Cloud Run+GPU =
 serving model**; NVDEC lever) . `ADR-013` (**GCP sole compute stack**) . **? ADR-014** (this).
 Non-ADR: realizes `PLATFORM\_ARCHITECTURE.md`'s `ComputeBackend`/`StorageBackend` registries; the
 truly-local profile = `VIDEO\_LOCALITY\_MODEL.md` L0 + `DESKTOP\_APP\_PLAN.md` `LocalDesktop`;
 informed by the archived `DEPLOY\_REPORT\_GCP\_2026-06-20` (cross-GPU sub-pixel parity finding).
 707:- Aligns with ADR-012 (GPU-native) + ADR-013 (GCP sole stack) + the 2026-06-20 deploy
 report; the truly-local profile preserves the ADR-009 privacy/offline self-host story.

10. user (2026-06-21T13:54:20.717Z)

Cloud Run GPU ****Job**** image for the rally worker (COMPUTE_DECOUPLED_SERVING)

 Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co
 CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the own
 M7 real run (the \$100/?10k budget, D17); CI does NOT build this (a CUDA image is too
 dispatch shim is proven with a fake client (tests/test_compute_backend.py).

 Two python stacks, deliberately separate (a framework conflict otherwise ? same split
 native GPU box, deploy/digitalocean/gpu_setup.sh):

- * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
- * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
 (the native runner shells out to \$WASB_PYTHON), so the app's torch never co-installs

 ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance
 commercial sharing. They are NOT baked into this image; stage them at runtime (a b
 mount) and point \$WASB_WEIGHTS at them. Resolve WASB-C3 before any public, n

FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04

```
ENV DEBIAN_FRONTEND=noninteractive \
  PYTHONUNBUFFERED=1 \
  PIP_NO_CACHE_DIR=1 \
  CONDA_DIR=/opt/conda \
  WASB_REPO_DIR=/opt/models/WASB-SBDT
```

ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB en

```
RUN apt-get update && apt-get install -y --no-install-recommends \
  ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
  python3 python3-pip python3-venv \
  && rm -rf /var/lib/apt/lists/*
```

--- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified torch-

```
RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
  && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
  && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkg/main || true \
  && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkg/r || true \
  && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
  && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
```

```

    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
        torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
    && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python" -m pip
install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
        hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml matplotlib
"numpy<1.24"

```

--- Main application env ---

```

WORKDIR /app
COPY pyproject.toml README.md ./
COPY backend ./backend
COPY training ./training
RUN python3 -m pip install --upgrade pip && python3 -m pip install .

```

The native runner finds the WASB env + repo by env (no `conda activate` needed); we runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stay

```

ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
    WASB_REPO_DIR=/opt/models/WASB-SBDT \
    WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
    RALLY_DETECTOR_IMPL=native

```

The dispatcher (CloudRunCompute) supplies the per-execution args:

infer --video-uri gs://? --out-uri gs://? --done-uri gs://? --segmenter wasb_hybrid --set

```

ENTRYPOINT ["python3", "-m", "backend.worker"]
=====README=====

```

Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6)

The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud\_run.py`)
dispatches a Cloud Run ****Job**** execution running this image's worker on an ****L4****, then
****bucket-polls****
a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.

```

> **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real build +
push +
> run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ? confirm cmd +
hourly
> cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does **not** build
this image.

```

0. Prerequisites (owner)

- A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs enabled.
- GPU quota for L4 (`GPUS\_ALL\_REGIONS` ? 1) in the chosen region (asia-south1, else Singapore ? see
`deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
- The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb\_badminton\_best.pth.tar`.

1. Build + push the image (Artifact Registry)

```

` ``bash
PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest -f
deploy/cloudrun/Dockerfile .
` ``

```

(A CUDA + conda image is large; the first build is slow. Budget-watch per D17.)

2. Create the Cloud Run **Job** (GPU, scale-to-zero)

```
```bash
gcloud run jobs create rally-worker \
 --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
 --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
 --max-retries 0 --task-timeout 3600 \
 --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
```

(stage the weights into the container at start, or fetch from gs:// in an init step ? WASE

```
```
```

3. Wire the dispatcher (control plane)

The control plane runs with the **cloud-serving** profile (``DEPLOYMENT_PROFILE=cloud-serving`` ? ``compute_target=cloud_run``). ``get_compute_backend(config)`` then builds ``CloudRunCompute`` from env:

```
```bash
export DEPLOYMENT_PROFILE=cloud-serving
export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION GOOGLE_CLOUD_PROJECT=$PROJECT
```

**A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job in running in-process; the container runs the SAME worker INLINE (compute\_target for**

```
python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri gs://<bucket>
--segmenter wasb_hybrid
```
```

4. M7 acceptance (the owner runlog)

Capture a ``DEPLOY_REPORT_cloud-serving_<date>.md`` under ``docs/COMPUTE_DECOUPLED_SERVING/runlogs/`` (mirrors the truly-local template): the exact commands, rally counts, ``gpu_active_seconds`/`wall_seconds`/`bytes_out``, a reel smoke-test (``ffmpeg -f null -`` exit 0 + a 5-frame contact sheet), gotchas, and the **\$\$0** teardown audit.

Gotchas

- The dispatched container **must run inline** ? ``CloudRunCompute`` forces ``compute_target=local_gpu`` + ``indexing.detector_impl=native`` in the per-execution args so the Job never re-dispatches itself.
- Completion is a **bucket marker** (``<out_uri>/_dispatch/<token>.done``), not the Cloud Run API status ? so it reuses the storage layer + ``execution_policy.poll_until`` and survives a control-plane restart.
- A **failed** Job currently writes **no** marker ? the dispatcher's bounded poll times out (``ExecutionPolicy.max_wait_sec``) and ``_dispatch_remote`` returns a clean **rc 4** (not a traceback).
A failure-marker (carrying the real non-zero rc, so the dispatcher distinguishes **failed** from **hung/cold-start-slow**) is a follow-up.
- libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default libx264.

11. assistant (2026-06-21T13:54:23.157Z)

I have everything needed. Let me read the ADR-012 and ADR-014 bodies to capture the precise serving-model ratification and the owner gates.

12. user (2026-06-21T13:54:24.219Z)

```
598      ## ADR-012: Compute = GPU-native (L4/T4); TPU deferred; Cloud Run + GPU (scale-to-zero)
is the supported serving model
599
600      **Date:** 2026-06-20 . **Status:** RATIFIED (owner, in-session). **Refines ADR-011**
(concretizes the "GCP = serving" path).
601
602      ### Decision
```

```

603
604     1. **Stay GPU-native (NVIDIA L4/T4) for all compute** ? local box, GPU VM, and the
serving path ?
605         for the foreseeable roadmap. **Do NOT invest in TPU now.** Revisit TPU **only**
if/when a large
606         *owned* transformer (Qwen3-VL Gen-2+, owner-gated) at *sustained batch* scale becomes
the dominant
607         cost, and **only after a benchmark** proves it ? never speculatively.
608     2. **Cloud Run + GPU (L4), scale-to-zero, is the supported SERVING deployment model**
for the
609         pay-per-video orchestration economics ("users pay us to orchestrate on our GCS bucket
+ VMs /
610         Cloud Run"): pay-per-request GPU-seconds = the exact per-video bill, zero idle-GPU
cost, co-located
611         with `gs://khelsutra-rally-corpus`. This is the concrete form of ADR-011's deferred
**M3/M4**.
612     3. **The named optimization lever is GPU-accelerated video decode (NVDEC)**, not more
GPU FLOPs ? the
613         measured bottleneck is video *decode*, not the model.
614
615     ### Rationale
616
617     - **Workload isn't TPU-shaped.** WASB is a tiny (5.8 MB) PyTorch CNN doing per-frame
heatmap
618         inference; the owned model is a VLM. TPUs win on large, dense, *batched* matmul at
training scale;
619         streaming per-frame CNN inference with small/dynamic batches is a poor fit.
620     - **The real bottleneck is video decode, which TPU does nothing for.** Measured on the
2026-06-20 GCP
621         L4 M2 run: cv2 PNG extraction at ~12 fps with the **GPU idle during extraction**. The
lever that
622         helps is **NVDEC** (hardware decode on L4/T4) + parallel decode ? i.e. lean *into* the
GPU.
623     - **Portability / single-runtime.** TPU needs XLA (torch_xla / JAX) ? significant
fragile NRE to port
624         WASB (torch 1.11) + the pipeline, a second runtime to maintain, and **GCP-only** ? it
would break
625         the local ? VM ? Cloud Run parity the DECOUPLED storage/compute seams exist to
provide. One portable
626         PyTorch+CUDA image that runs identically everywhere is worth more than peak FLOPs
here.
627     - **Economics.** Cloud Run+GPU scale-to-zero meters per request = clean per-video
billing; TPU pricing
628         (v5e pods / queued / committed-use) targets sustained training and its regional
availability is
629         *narrower* than GPU (the M2 run already hit a Mumbai GPU stockout ? landed in
Singapore; TPU would be
630         worse).
631
632     ### Consequences
633
634     - The **deferred M3 (inference serving endpoint + per-video billing) and M4 (scale-to-
zero + cost
635         guard)** from ADR-011 become a **new tracked subproject ? "Cloud Run + GPU serving"***
? to be spun
636         up (as a `docs/PROJECT_DOC_TEMPLATE.md`-based doc) **AFTER `docs/DECOUPLED_COMPUTE/`
is archived**
637         (owner sequencing). Seed scope below. **? REALIZED in ADR-014 (2026-06-21):** the
subproject is
638         `docs/COMPUTE_DECOUPLED_SERVING/`, **renamed** to reflect that **truly-local is a co-
equal profile**
639         (not just cloud serving); ADR-014 expands this 7-point seed into the M0?M10 plan.
640     - **First subproject work item = containerize `python -m backend.worker`** for Cloud
Run. This is the
641         known hard part: WASB's torch-1.11 conda env makes the image non-trivial (two runtimes

```

```

? the
642     indexer venv + the `wasb` conda env). The decoupled storage/compute seams + the native
runner
643     (`detector_impl="native"`, PR #256) make the *code* portable; packaging is the gap.
644     - Keep the **storage/compute decoupling seams** intact so the SAME container runs local
/ GPU VM /
645     Cloud Run (the per-video workspace convention, in progress, reinforces this).
646     - **Edge auth is a hard prerequisite** before any public serving (engine API is CORS
`*`, no auth) ?
647     carried from the appliance track.
648     - **Region/capacity reality:** GPU stockouts are common (Mumbai was fully stocked-out on
2026-06-20);
649     scale-to-zero + multi-region/flexible-zone selection is part of why Cloud Run+GPU
fits.
650     - TPU is explicitly **out of scope**; this ADR records *why* so it isn't relitigated.
651
652     ##### Seed scope for the "Cloud Run + GPU serving" subproject (formalize post-archival)
653     1. Containerize `backend.worker` (indexer venv + `wasb` conda env; NVDEC-capable base
image).
654     2. Cloud Run + GPU (L4) service: `infer` over a `gcs://` workspace; scale-to-zero.
655     3. Per-video cost ledger (on the `jobs` table) = the per-customer bill (M3).
656     4. Cost guard + concurrency caps (M4).
657     5. Edge auth (tenancy boundary) before public exposure.
658     6. NVDEC GPU-decode to kill the extraction bottleneck.
659     7. DPA / consent gate for non-owner uploads (productization, not compute).
660
661     ## ADR-013: Drop DigitalOcean for compute ? GCP is the SOLE compute stack (training +
serving)
662
663     **Date:** 2026-06-20 . **Status:** RATIFIED (owner, in-session: "drop DO and document
it? record it and bake it for all future work"). **Supersedes the DO=training split in ADR-011;
refines ADR-010 + ADR-012.**
664
665     ### Decision
666
667     1. **DigitalOcean is DROPPED as a compute provider.** **GCP is the sole compute stack**
for BOTH training and serving. (ADR-011's "DO=training, GCP=serving" split is retired ? GCP does
both. The DECOUPLED scope decision in ADR-011 ? M0?M2 + M3.5 ? still stands.)
668     2. **Standing default for ALL future compute work = GCP** (a GPU VM today via
`deploy/gcp/`; Cloud Run+GPU for serving per ADR-012). Do **not** re-evaluate DO/Paperspace for
compute without a new ADR.
669     3. The `$200` DO credits, if used at all, go **only to non-compute** that DO bills
cleanly (CPU droplets / a site host / Spaces storage / bandwidth) ? **never GPU**.
670
671     ### Rationale (researched 2026-06-20, against DO's own docs)
672
673     - **DO GPU is neither enabled on the account nor a clean free-credits path.** The DO API
`/v2/sizes` returns **0 GPU sizes** (GPU Droplets not enabled); GPU Droplets are **North-
America-only** (NYC2/TOR1/ATL1/RIC1/AMS3 ? no India/Singapore); and DO's own docs state that for
**newer customers DO charges the team's primary payment method** when GPU Droplets are created
(anti-abuse) ? so GPU spend hits the **card**, not the promo credit. The ADR-011 premise ("200
credits fund DO training") does **not** hold.
674     - **Paperspace (now DO Gradient) free GPU is subscription-gated** (a paid "G"/Growth
plan; high-end A100/H100 needs the $39/mo Growth plan), and Paperspace bills **separately** from
DO Droplet credits ? the $200 credit doesn't fund it either.
675     - **GCP works today:** quota granted (`GPUS_ALL_REGIONS=1`), bucket + SA + `$100` budget
provisioned, and M1/M2/M3.5 ran on an L4 for ~$12/run with no subscription. Real money, but
trivial per run ? and **one** portable PyTorch+CUDA stack (local ? GPU VM ? Cloud Run), no
second provider to maintain.
676
677     ### Consequences
678
679     - **Compute provisioning = `deploy/gcp/` only.** `SETUP_NEW_MACHINE.md` repointed to
GCP; `deploy/digitalocean/README.md` marked **DEPRECATED** ; `HOSTING_PLAN.md`'s "All-
DigitalOcean" compute option retired.

```

680 - **``deploy/digitalocean/` scripts are KEPT` ? `bootstrap.sh` / `mount\_scratch.sh` / `gpu\_setup.sh` are **``provider-agnostic``** Linux-GPU setup the GCP runbook reuses; only the DO-specific **``provisioning``** (doctl droplet create, Spaces) is dropped. (Optional later cleanup: move them to `deploy/common/`.)**

681 - The shared DO CPU droplet **``claude-do-rally-test``** (another session's testbed) is **``owner-coordinated``** cleanup, separate from this decision; the freshly-rotated **``DO API token``** can be retired once that droplet is gone.

682 - **``Storage stays bucket-abstracted``** (`gcs://` primary). The `s3` backend still speaks any S3-compatible store, so **``DO Spaces remains a *possible* storage target``** even though DO **``compute``** is dropped.

683 - **``Capacity reality:``** GCP GPU stockouts are common (asia-south1/Mumbai was fully stocked-out 2026-06-20 ? the M2 run landed an L4 in asia-southeast1/Singapore; weights still land in the asia-south1 bucket). Sweep zones/regions; scale-to-zero (ADR-012) is part of why Cloud Run+GPU fits.

684 - **``DECOUPLED\_COMPUTE``** archives on M2 completion with a note that the DO leg was **``dropped``** (this ADR), not "delivered on DO".

685

686 **## ADR-014: Compute-decoupled rally serving ? one pure core, profile-selected compute + storage**

687

688 **``Date:``** 2026-06-21 . **``Status:``** ? **``RATIFIED** (owner, 2026-06-21 ? M0 design sign-off; build M1+). **``Refines ADR-011 + ADR-012``** (it **``realizes``** their deferred M3/M4; it does **``not``** supersede them); **``builds on ADR-013``** (GCP sole compute stack). Full design: ``docs/COMPUTE_DECOUPLED_SERVING/``.

689

690 **### Evolution / lineage (trace the idea)**

691 ``ADR-005`` (permissive licensing ? engine + image stay MIT/Apache/BSD) . ``ADR-008`` (train==serve featurizer single-sourcing ? the parity discipline this extends to local?cloud) . ``ADR-009`` (**``local-first delivery``** ? origin of the truly-local profile) . ``ADR-010`` (DECOUPLED DO?GCP) . ``ADR-011`` (**``deferred M3 serving + M4 billing``**; overrode "rent nothing / not a service"; \$06 gates = DPA/consent, collector `md5`, WASB-C3) . ``ADR-012`` (**``Cloud Run+GPU = serving model``**; NVDEC lever) . ``ADR-013`` (**``GCP sole compute stack``**) . **``? ADR-014``** (this). Non-ADR: realizes ``PLATFORM_ARCHITECTURE.md``'s ``ComputeBackend``/``StorageBackend`` registries; the truly-local profile = ``VIDEO_LOCALITY_MODEL.md`` L0 + ``DESKTOP_APP_PLAN.md`` ``LocalDesktop``; informed by the archived ``DEPLOY_REPORT_GCP_2026-06-20`` (cross-GPU sub-pixel parity finding).

692

693 **### Decision**

694

695 1. The rally engine is a **``PURE function of (input bytes + config) ? outputs``**, in three deterministic stages ? **``DETECT``** (``DetectorRunner.run_predict`?CSV`), **``WINDOW``** (``trajectory_to_action_windows``, CPU-pure), **``STITCH``** (``VideoCompiler.compile_highlights``, ffmpeg) ? that are **``NEVER branched on the deployment profile``**.

696 2. A **``ComputeBackend``** / deployment profile decides **``WHERE``** the core runs (local process vs a Cloud Run GPU job); a **``StorageBackend``** decides **``WHERE I/O comes from``** (local FS / USB / mounted Google Drive / bucket). **``Both are selected by ONE startup/setup config``** (``deployment.profile`` + ``compute_target``), realizing the ``PLATFORM_ARCHITECTURE`` ``local|hosted|by_worker`` registry.

697 3. Ship at least two profiles: **``truly-local``** (local GPU + **``in-process stitch``**, no cloud) and **``cloud-serving``** (a Cloud Run GPU job runs detect **``AND``** stitch), plus **``cpu-mock``** for CI and a reserved, **``deferred``** ``by_worker`` hybrid.

698 4. In cloud-serving, **``STITCH runs on Cloud Run``** (co-located with detect) so its CPU/wall-time + egress are **``metered``** into a per-job cost ledger alongside ``gpu_active_seconds`` ? stitching is compute-intensive and its cost must be accounted for. (Truly-local stitch is intentionally **``unmetered``**.)

699 5. **``Parity``** is enforced at **``WINDOW + STITCH``** (byte-exact for a fixed trajectory); **``DETECT``** is byte-exact only **``same-GPU``** ? cross-GPU is sub-pixel-different (deploy report 2026-06-20), so it is judged at the **``rally/window level``**, never CSV bytes.

700 6. Land **``smallest-safe-first, one PR per milestone (M0?M10)``**, **``DEFAULT-OFF``** where a change touches core callers (e.g. the configurable output base), each guarded by golden-fixtures + a **``profile-parity test``**; default flips ship a **``Launch Report``**.

701

702 **### Rationale**

703

704 - The seams already exist: ``_resolve_runner`` is the single compute seam, the ``StorageBackend`` ABC + 4 backends + the worker fetch/put are built, and the stub detector

(`replay\_csv`) gives a ****\$0 deterministic**** core. Realizing a profile system is ****composition,**
not a rewrite******.

705 - Separating ****WHERE**** (ComputeBackend, coarse, per-job) from ****HOW**** (DeviceContext,
fine, per-GPU) from ****WHAT-bytes**** (StorageBackend) keeps each axis independently testable and
avoids the main trap ? conflating them.

706 - ****Metering stitch on Cloud Run**** is the only way the owner's real serving cost (GPU +
CPU re-encode + egress) is attributable per job; stitch-on-laptop hides it.

707 - Aligns with ADR-012 (GPU-native) + ADR-013 (GCP sole stack) + the 2026-06-20 deploy
report; the truly-local profile preserves the ADR-009 privacy/offline self-host story.

708

709 **### Consequences**

710

711 - ****Positive:**** one core, provably identical across modes; CI proves the decoupling for
****\$0**** (stub + storage fakes + a profile-parity test); new compute targets (byo_worker,
SkyPilot, Modal) slot into the enum without core changes; ****cost becomes visible per job****
(incl. stitch).

712 - ****Cost/new surface:**** a new `deployment`+`compute\_target` config surface, a Cloud Run
****dispatch shim****, a ****containerized GPU+ffmpeg worker image****, a ****cost-ledger migration****, and
****edge auth**** must all be built + maintained. Cross-GPU detection is ****not**** byte-identical ?
parity asserted at the window level. The `byo\_worker` tier carries a real ops/support burden
(deferred, power-user only).

13. assistant (2026-06-21T13:54:26.661Z)

I now have the complete picture across the README tracker, the runlogs template, the
Dockerfile/README, and ADR-011/012/013/014. I have everything needed to assemble the structured
answer.

14. user (2026-06-21T13:55:29.787Z)

Output does not match required schema: root: must NOT have additional properties

15. assistant (2026-06-21T13:55:31.707Z)

I added an extra property. Let me resubmit with only the schema-defined fields.

16. user (2026-06-21T13:56:25.657Z)

Structured output provided successfully

17. assistant (2026-06-21T13:56:27.153Z)

Structured output submitted successfully.